Lec 04 Value Iteration & Policy Iteration

Value iteration algorithm
- Algorithm description
- Elementwise form
- Pseudocode

Policy iteration algorithm
- Algorithm description
- Elementwise form
  - Step 1: Policy evaluation Details
- Step 2: Policy improvement Details
- Pseudocode

Truncated policy iteration algorithm
- **Compare value iteration and policy iteration**
- Pseudocode

Value iteration algorithm

之前已经介绍了 ***Bellman Optimality Equation***:

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

并且，我们也可以通过 ***contraction mapping theorem*** ，通过迭代的方法求解:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 1, 2, 3, \ldots$$

其中 v_0 可以是随机的.

这种方法就称为 ***value iteration.***

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 1, 2, 3, \ldots$$

Algorithm description

整个算法可以分为两步:

- **Step 1: policy update**
  这一步就是通过求解右侧的优化问题，得到一个新的 policy

  $$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

  其中 v_k 给定

- **Step 2: value update**
  这一步就是在上一步的基础上，通过迭代得到 v_{k+1}

  $$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

注意：在这里 v_k 并不是 state value! 只是过程中间满足 Bellman Equation 的一个向量而已，它可以是任何向量！

Elementwise form

- **Step 1: Policy update**

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

该公式的 **elementwise form**为：

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')\right)}_{q_k(s,a)}, \quad s \in \mathcal{S}$$

求解上述优化问题的 optimal policy 为：

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^(s) \\ 0 & a \neq a_k^(s) \end{cases}$$

其中 $a_k^*(s) = \arg\max_a q_k(a,s)$。$\pi_{k+1}$ 被称为 **greedy policy**，因为它仅仅选择最大的 q-value。

- Step 2: Value update

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

该公式的 **elementwise form**为：

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')\right)}_{q_k(s,a)}, \quad s \in \mathcal{S}$$

因为π_{k+1} 是 greedy 的，上述等式可以简化为：

$$v_{k+1}(s) = \max_a q_k(a,s)$$

Pseudocode

$v_k(s) \to q_k(s,a) \to \text{greedy policy } \pi_{k+1}(a|s) \to \text{new value } v_{k+1} = \max_a q_k(s,a)$

Pseudocode: Value iteration algorithm

Initialization: The probability model $p(r|s,a)$ and $p(s'|s,a)$ for all (s,a) are known. Initial guess v_0.

Aim: Search the optimal state value and an optimal policy solving the Bellman optimality equation.

While v_k has not converged in the sense that $\|v_k - v_{k-1}\|$ is greater than a predefined small threshold, for the kth iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

q-value: $q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')$

Maximum action value: $a_k^*(s) = \arg\max_a q_k(a,s)$

Policy update: $\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

Value update: $v_{k+1}(s) = \max_a q_k(a,s)$

Policy iteration algorithm

Algorithm description

给定一个随机的 **initial policy** π_0,

- **Step 1: Policy evaluation (PE)**
  这一步是为了计算π_k 的 **state value**

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  注意v_{π_k} 是一个 **state value function**。

- **Step 2: Policy improvement (PI)**

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

  这个最大化是 **componentwise** (逐分量) 进行的!

Elementwise form

Step 1: Policy evaluation Details

- Matrix-vector form:

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$$

- Elementwise form:

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S}$$

当 $j \to \infty$ 或j足够大，或者$\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\|$足够小时停止。

Step 2: Policy improvement Details

- **Matrix-vector form:**

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

- **Elementwise form**:

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')\right)}_{} q\pi_k(s,a), \quad s \in \mathcal{S}$$

这里，$q_{\pi_k}(s,a)$是 **policy** π_k 下的 **action value**。
令

$$a_k^*(s) = \arg\max_a q_{\pi_k}(a,s)$$

那么，**greedy policy** 为

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^(s), \\ 0 & a \neq a_k^(s). \end{cases}$$

Pseudocode

Pseudocode: Policy iteration algorithm

Initialization: The probability model $p(r|s,a)$ and $p(s'|s,a)$ for all (s,a) are known. Initial guess π_0.

Aim: Search for the optimal state value and an optimal policy.

While the policy has not converged, for the kth iteration, do

Policy evaluation:

Initialization: an arbitrary initial guess $v_{\pi_k}^{(0)}$

While $v_{\pi_k}^{(j)}$ has not converged, for the jth iteration, do

For every state $s \in \mathcal{S}$, do

$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s')\right]$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$q_{\pi_k}(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')$

$a_k^*(s) = \arg\max_a q_{\pi_k}(s,a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

Truncated policy iteration algorithm

Compare value iteration and policy iteration

Policy iteration: start from π_0 (从π_0 开始)

- Policy evaluation (PE):

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- Policy improvement (PI):

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

Value iteration: start from $v_0 \to v^*$ (从 v_0 开始，收敛至 v^*)

- Policy update (PU):

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

- Value update (VU):

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

这两个算法非常相似:

Policy iteration: $\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$

Value iteration: $u_0 \xrightarrow{PU} \pi_1' \xrightarrow{VU} u_1 \xrightarrow{PU} \pi_2' \xrightarrow{VU} u_2 \xrightarrow{PU} \dots$

- PE = policy evaluation
- PI = policy improvement
- PU = policy update
- VU = value update

考虑求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$的步骤:

$$v_{\pi_1}^{(0)} = v_0$$

$$\begin{aligned}
& v_{\pi_1}^{(0)} = v_0 \\
\text{value iteration} \leftarrow v_1 \longleftarrow & v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& \vdots \\
\text{truncated policy iteration} \leftarrow \bar{v}_1 \longleftarrow & v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& \vdots \\
\text{policy iteration} \leftarrow v_{\pi_1} \longleftarrow & v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}$$

- **Value iteration algorithm** 只计算 一次。
- **Policy iteration algorithm** 计算无数次迭代。

- **Truncated policy iteration algorithm** 计算有限次迭代，比如j次。剩下的从j到 ∞ 的迭代被truncated了。

Pseudocode

Pseudocode: Truncated policy iteration algorithm

Initialization: The probability model $p(r|s,a)$ and $p(s'|s,a)$ for all (s,a) are known. Initial guess π_0.

Aim: Search for the optimal state value and an optimal policy.

While the policy has not converged, for the kth iteration, do

Policy evaluation:

Initialization: select the initial guess as $v_k^{(0)} = v_{k-1}$. The maximum iteration is set to be j_{truncate}.

While $j < j_{\text{truncate}}$, do

For every state $s \in \mathcal{S}$, do

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a|s)\left[\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k^{(j)}(s')\right]$$

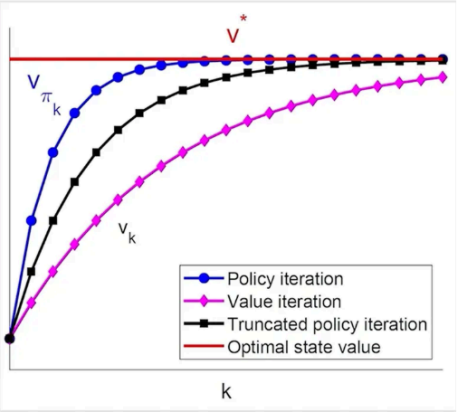
Set $v_k = v_k^{(j_{\text{truncate}})}$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$$q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')$$

$a_k^*(s) = \arg\max_a q_k(s,a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise